

Formal Methods and Specification (SS 2025/26)

Lecture 1: Introduction

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Language/Jazyk

Bude-li přítomen aspoň jeden student, který neumí český,
bude přednáška **v angličtině**

Budete mít k dispozici **slajdy** v **obou** jazycích

Budu vděčný za jakoukoli informaci o pravopisných chybách.

Zeptejte se!

Nebude-li přítomen žádný student, který neumí český,
můžete si jazyk vybrat.

Contact Information

Always:

- ▶ Stefan Ratschan
- ▶ <http://www.cs.cas.cz/~ratschan>
(especially link “consultation hours”)
- ▶ stefan.ratschan@fit.cvut.cz

Before/after MI-FME:

- ▶ FIT, A-1029

Rest of the time:

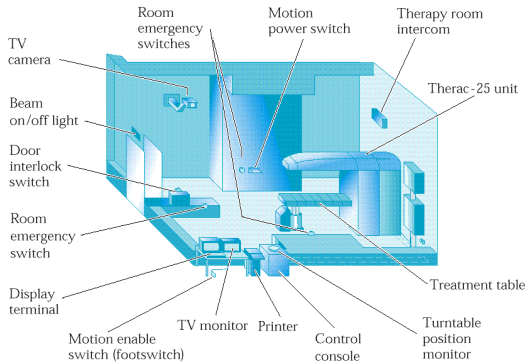
- ▶ Institute of Computer Science of the Czech Academy of Sciences
- ▶ Pod Vodárenskou věží 2
- ▶ Metro station Ládví

Today

- ▶ Introduction
- ▶ Organizational Matters

Now: a catastrophic beginning.

Therac-25 (1985)



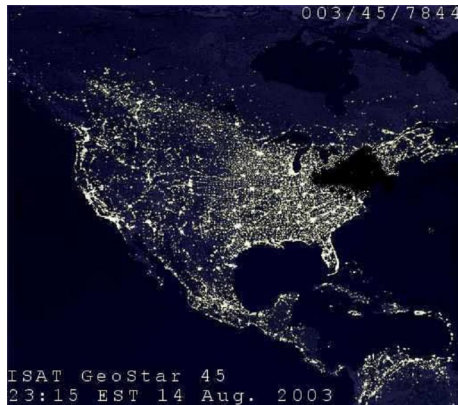
- ▶ Radiation therapy machine
- ▶ Original machine (therac-20) was almost entirely **mechanical**.
- ▶ Observation: a mechanical machine is dangerous (problems due to aging), so use **software** control
- ▶ Result: At least 6 **accidents** with over-radiation, some of them deadly.

Ariane 5 Flight 501 (1996)



- ▶ The flight ended with an **explosion** directly after its start
- ▶ Costs: 290 M€
- ▶ Program delayed by one year

Northeast Blackout of 2003



- ▶ **Power outage** in a large part of the U.S.A. 14.-16.8.2003
- ▶ More than **10 direct fatalities**
- ▶ Huge costs

So What?

Connection with computer science?

- ▶ Each of the Therac-25 overdoses: result of a **software bug** originally excluded by a mechanical device
- ▶ Ariane 5 explosion:
 - ▶ result of an **overflow** in float to integer conversion
 - ▶ two redundant computers (same problem in both)
 - ▶ software function not need by Ariane 5 (different from version 4)
- ▶ Northeast blackout: **software bug** → alarm system failure, with earlier alarm response, no blackout

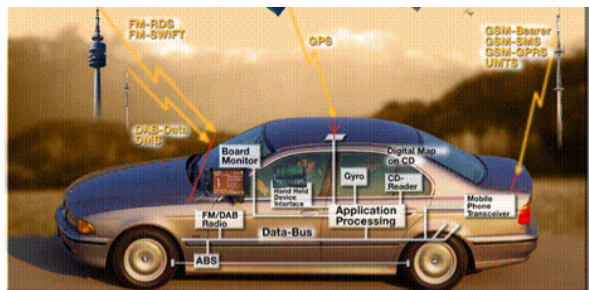
Bugs are often simple, but

- ▶ often occur in complicated unexpected situations, and
- ▶ may have catastrophic consequences because of complex chain reactions.

Difficult to **detect** by **testing**.

Cyber-Physical Systems (CPS)

Integration of computation and physical processes



“Cost of electronics in cars expected to move beyond 50% soon”
[Emb, 2005].

Most computing power today not in desktop computers

Safety Critical System

Not only,
 increasingly tight integration
 of computation and physical processes,
but also
 integration into daily **human life**

Extreme example: software implantable medical devices

Malfunction can **endanger** human life

Correct design essential

This course will contribute to this goal.

Formal Methods and Specification

Formal? **mathematical precision**

Example

```
result:= "sorted"
for i:= 1 to 9 do
  if a[i]>a[i+1] then
    result:= "unsorted"
  endif
endfor
return result
```

Is this algorithm **correct**?

- ▶ If we want to decide whether to cook dumplings or cabbage today?
- ▶ If we want to connect an iPad to the internet?
- ▶ If we want to know whether the array *a* is sorted?

Correctness of algorithm only makes sense
in combination with a **specification**

Specification

```
result:= "sorted"
for i:= 1 to 9 do
  if a[i]>a[i+1] then
    result:= "unsorted"
  endif
endfor
return result
```

Let's try to write a specification:

- ▶ Input: array a of length 10
- ▶ Output: if for all i in $\{1, \dots, 9\}$, $a[i] \leq a[i + 1]$ then "sorted"
else "unsorted"

Attention: **side effects!** timing

For now, algorithms without side effects: **I/O specification**

Well-Formed Specifications

Example:

- ▶ In: natural number k
- ▶ Out: a prime factor of k

I/O specifications

- ▶ may allow **more than one** correct **output** for one input, but
- ▶ must have **at least one output** for each input.

Specification Languages

Specifications are usually written in **natural languages**
(Czech, English, etc.)

English “eventually” vs. Czech “eventuálně”

anonymous Czech car manufacturer:

“each car will be connected to at most one mobile phone”

Problem:

- ▶ imprecision, ambiguity
- ▶ difficult to understand for computers

Solution: **artificial languages, predicate logic**

Further examples: see assignments

More precisely: first-order predicate logic +
set theory, theory of integers, lists, arrays etc.

Course Objective

Formal Methods and Specification

Answer grand question:

*How to use **mathematical precision** when*

- ▶ ***specifying** the behavior of programs, and*
- ▶ ***writing software**, that fulfills its specification?*

But: Is this not just a **theoretical** exercise, that is **irrelevant** in **practice**?

First Course Benefit: Brain Upgrade

The human brain developed a few **thousand years ago**, with a **different purpose**:



Due to this historical purpose, the brain module easily makes **mistakes** in modern, and especially computer science life.

By training mathematical precision when reasoning about programs, we **improve** our **brain** for computer science life

Concrete benefit for you: You will be **better software engineers**.

Second Benefit: Practical Techniques

Industry uses formal methods more and more

Semi-automatic partial correctness proofs of computer chips
routinely used in **hardware** industry [Kropf, 1999]
motivated by Intel FDIV bug

Software: Prove “**small**” properties [D’Silva et al., 2008]

- ▶ correctness of compiler transformations
- ▶ absence of division by zero
- ▶ absence of array overflow
- ▶ ...

especially in so-called safety-critical applications

The course will help you to
understand and use methods that are **used in practice**

Examples

- ▶ **Airbus** uses automatic methods for program verification
<http://www.absint.de/astree/>
- ▶ **Microsoft's** “Windows Driver Kit” includes formal verification tool
“Static Driver Verifier”
- ▶ Formal verification of **stock trading** algorithms <https://www.imandra.ai/>
- ▶ **BTC Embedded Systems AG**
<https://www.btc-es.de>
- ▶ **Amazon** web services
<https://doi.org/10.1145/2699417>
- ▶ **Facebook**
https://doi.org/10.1007/978-3-319-17524-9_1

Current research continuously **increases applicability**,
in the near future, they will be in common usage.

Third Benefit: Formal Methods are Beautiful



What this Course is Not About

Complete formalization of the software development process:

- ▶ B-method
- ▶ Z-notation

Despite many years of research still only used in very specific applications.

But: the course will also help you to understand such methods.

Prerequisites: Logic

Incomplete list of important notions: term, arity, formula, sentence, free/bound variables, substitution, interpretation, variable assignment, $\models$, validity, satisfiability, equivalence, equivalence rules

See

- ▶ basic undergraduate courses
- ▶ MI-TES: lecture “basics of practical logic”
- ▶ Lectures notes “Formal Modeling and Automatic Analysis of Complex Systems” (chapter 1, except for section “a complete proof calculus”)

Conventions:

- ▶ Tautology: $\top$, contradiction: $\perp$
- ▶ Quantifiers: $\exists x . p(x) \wedge q(x)$ means $\exists x . [p(x) \wedge q(x)]$
and not $[\exists x . p(x)] \wedge q(x)$
- ▶ Substitution of term t for the variable x : $A[x \leftarrow t]$
(e.g., $x + y[y \leftarrow x^2]$ is $x + x^2$)

Notation

FIT-notation

In practice: wide variety!

One of most important capabilities
ability to understand concepts independently from notation.

Proofs

Simple proofs: equivalence rules

Example: $\perp \Rightarrow A$ is equivalent to $\top$

(lecture notes, section 1.5)

Not everything can be proved on the basis of equivalence rules
(the method is not complete).

Next lecture: **general proof method**

Preliminary Lecture Plan

1. Introduction
2. A complete proof method
3. Logical theories and modeling of data structures
4. Correctness of Linear Program
5. Correctness of Programs With Control Structures
6. Function, procedures, object-oriented programming
7. Symbolic execution
8. ...

2-3: theoretical basis

4-: formal methods

Course Organization

*Aurea prima sata est aetas, quae vindice nullo,
sponte sua, sine lege fidem rectumque colebat.
Poena metusque aberant nec verba minantia fixo
aere legebantur, nec supplex turba timebat
iudicis ora sui, sed erant sine vindice tuti.*

... education in the past ...

Traditional Education

Conditions:

- ▶ external **memory** (i.e., books) **expensive** and difficult to access
- ▶ **few** important **documents** (Homer, Platon, Ovid, bible)
- ▶ almost **no changes** of scientific **knowledge** during one's lifetime
- ▶ **computer** = a **person** that computes
- ▶ people in the hands of **one ruler** (emperor, leader, party, etc.), truth controlled by **authorities** (church, party, etc.)

Those conditions are **gone**,
but traditional education not.

Year 2025/26

- ▶ external memory (i.e., books) expensive and difficult to access
*All books written before 1900 can be stored on **one disk***
- ▶ limited amount of important documents (Homer, Platon, Ovid, bible)
*Without even leaving our bed we have access to **more information** than our grand-parents during their whole life*
- ▶ almost no changes of scientific knowledge during one's lifetime
*Common technologies become **obsolete** within a few years, **innovation** is key to prosperity (innovation society)*
- ▶ computer = a person that computes
*One mobile **phone** computes **faster** than all computers (i.e., people) of the 19th century*
- ▶ people in the hands of one ruler (emperor, leader, party, etc.), truth controlled by authorities (church, party, etc.)
*Every **individual** bears **responsibility** for society, the political system needs **independent thinking, critical and courageous** people*

Consequences

- ▶ The problem is not information access, but abundance of information.
Instead of primitive memorization, we need the ability to process and organize information.
- ▶ Whatever knowledge we learn, we will need something else.
Key is the ability to orient oneself in new situations
- ▶ Almost everything becomes outdated in short time.
We must concentrate on a few essential things that will stay relevant through our whole life, on abilities instead of knowledge.
- ▶ It does not make sense to blindly learn algorithms (e.g., integration, solving linear systems of equations).
Concentrate on things beyond the abilities of computers: creativity, innovation, independent (but still precise) thinking
- ▶ We need abilities of understanding, discussion, and communication

Concrete consequences for this course?

Final Exam

Open book: You will be allowed to use any book, printouts of slides etc.

- ▶ “o.k., so I’ll just relax during the semester,
at the exam I will anyway be able to find the answers in books”
- ▶ “na cvika asi budu muset, ale **na přednášku kašlu**”

This will not work!

Two types of problems to solve at the final exam:

- ▶ Problems of the same or similar type
as the assignments during the semester.
- ▶ Problem, that test whether the student is able to apply the theory from the
lecture on his/her own in a **new** context

... which is difficult!

But if you work during the whole semester, then less so.

Work During the Semester

Typical didactical steps:

1. small example in lecture
2. theory (generalization of the example)
3. homework: bigger example
4. tutorial: check, discuss

Everything builds on the **lecture!**

If step 1 is unclear to you, you will not be able to do the rest!

Especially, you will have problems with doing your homework

There will be no time during tutorials to
explain the lecture to people who did not attend it

Ask, ask, ask, ask, ask ...!

Sources

Slides online (courses)

I will **not** follow a certain textbook

On the slides of the individual lectures I will provide references to sources

Lecture notes (under development)

You will not need them if you attend the lectures

If you miss some lectures, you will need additional sources

Lectures slides will not contain all information,

I will draw examples, figures etc. onto the whiteboard

Be careful with Wikipedia!

- ▶ A large part of the lecture material is not there.
(this is a master's course, where more advanced things are taught)
- ▶ If it is there, it often contains mistakes, bad explanations etc.

Help Needed

Feedback!

During the semester, I can still react ...

Literature I

Study of worldwide trends and R&D programmes in embedded systems in view of maximising the impact of a technology platform in the area. Report for the European Commission, 2005.

V. D'Silva, D. Kroening, and G. Weissenbacher. A survey of automated techniques for formal software verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(7):1165–1178, July 2008.

Thomas Kropf. *Introduction to Formal Hardware Verification*. Springer, 1999.